

Parte 2 - Architettura dei microservizi

L’**Architettura dei Microservizi** è un approccio alla creazione di un’applicazione server come un insieme di piccoli servizi indipendenti. Questo paradigma contrasta con l’architettura monolitica, dove l’intera applicazione è costruita come un’unica unità.

Microservizi vs. Monolitico

La differenza chiave risiede nell’organizzazione e nella gestione dell’applicazione.

Architettura Monolitica

In questa architettura, tutte le funzioni risiedono in un unico blocco.

Categoria	Caratteristica
Sviluppo e Deploy	Un'unica unità. Una modifica richiede la ricompilazione e il <i>deploy</i> dell'intero codice.
Componenti	Difficilmente separabili e logicamente autonomi.
Scalabilità	Non è possibile scalare i singoli componenti.
Dati	Atomicità garantita nativamente dal DB centrale.
Vantaggi	Design e progettazione da zero più semplici, <i>debug</i> più agevole, meno <i>overhead</i> .
Svantaggi	Maggiore complessità complessiva, avanzamento tecnologico lento, <i>Continuous Delivery</i> difficile, poca flessibilità.

Architettura a Microservizi

L’applicazione è suddivisa in un set di servizi più piccoli e disaccoppiati.

Categoria	Caratteristica
Sviluppo e Deploy	Piena autonomia di sviluppo, distribuzione e scalabilità per ogni servizio.
Componenti	Netta distinzione delle funzionalità e libertà tecnologica (più linguaggi/tecnologie).
Scalabilità	Possibilità di Scalabilità Orizzontale (più istanze dello stesso servizio) e Verticale (potenziamento hardware).
Dati	Ogni microservizio è proprietario dei suoi dati (sovranità dei dati) e della logica di dominio. L'atomicità tra operazioni che coinvolgono più servizi non è garantita nativamente (transazioni ACID non native).
Vantaggi	Resilienza incrementata, chiara distinzione delle competenze, flessibilità nell'aggiunta di nuove funzionalità.
Svantaggi	Generalmente maggiore complessità, <i>debug</i> e coordinamento degli aggiornamenti più complessi.

Comunicazione e indipendenza

I microservizi comunicano tra loro. La comunicazione avviene spesso tramite **Web API** esposte su server HTTP(S), utilizzando protocolli come **REST** (solitamente JSON, metodi HTTP) o **GraphQL** (il client decide quali dati leggere).

- **Endpoint:** un servizio web espone uno o più endpoint per la comunicazione.
- **API Gateway:** è il controllo del traffico per le richieste. Gestisce la mappatura degli endpoint al microservizio specifico, il **versionamento** delle API e facilita il *deploy* degli aggiornamenti senza *downtime*.

L'obiettivo teorico è che un crash di un microservizio non comprometta gli altri, mettendo *offline* solo le funzionalità di sua responsabilità. Questo può avvenire se i servizi non comunicano, o se sono in grado di continuare con i dati in loro possesso.

Dimensionamento di un microservizio

L'idea è che debba avere dimensioni contenute e lavorare in architetture a container per favorire la scalabilità orizzontale.

Si possono utilizzare i seguenti indicatori (da valutare con elasticità e in contesto):

1. **Numero di oggetti del modello dati:** un microservizio con troppe entità relazionali potrebbe avere troppe responsabilità. Un rapporto elevato tra gli oggetti del microservizio e gli oggetti totali dell'applicazione suggerisce una probabile dimensione eccessiva.
2. **Numero API esposte:** un numero eccessivo di API può indicare una quantità eccessiva di responsabilità.
3. **Tempi di realizzazione stimati:** un tempo di implementazione eccessivamente lungo può suggerire che il microservizio abbia dimensioni eccessive, anche se la complessità funzionale e algoritmica va considerata.

Una dimensione errata può portare alla proliferazione di microservizi troppo piccoli con eccessive interdipendenze, o microservizi troppo grandi che compromettono l'indipendenza.

Dalla teoria alla pratica

La teoria vuole che i microservizi siano autonomi e indipendenti e che il crash di uno di essi non comprometta il funzionamento degli altri.

Scenari di indipendenza reale

L'indipendenza totale può avvenire solo in due casi:

- **Assenza di comunicazione:** tutti i microservizi non comunicano tra loro e posseggono tutte le informazioni utili per il proprio compito. Questo è considerato non realistico o rarissimo in applicazioni complesse, poiché significherebbe avere silos sigillati come applicazioni monolitiche.
- **Copia locale dei dati (compromesso realistico):** i microservizi comunicano, ma ciascuno è in grado di continuare il proprio compito con i dati attualmente in suo possesso (magari una copia locale del dato richiesto al servizio *offline*). Questo scenario è realistico e verrà approfondito con *pattern* e tecniche specifiche (come la comunicazione asincrona per la replica dei dati).

Minimizzare l'interdipendenza

Come per il dimensionamento, non esiste un metodo efficace per tutti gli scenari. L'architettura corretta è quella che cerca di minimizzare l'interdipendenza tra i vari servizi, dimensionandoli correttamente e separando le responsabilità in modo opportuno.

Endpoint e Web API

Gli **endpoint** permettono di comunicare con un servizio web, e ogni servizio espone uno o più di essi.

Le **Web API** sono generalmente esposte su server HTTP(S) e possono essere pubbliche o private. I formati e i protocolli principali sono:

Tipo di API	Caratteristiche	Formato	Struttura	Implementazioni .NET
REST (REpresentational State Transfer)	Utilizza i metodi HTTP (GET, POST, PUT, DELETE, ecc.).	JSON, XML	Libera a discrezione dell'implementazione.	Integrato in ASP.NET Core.

SOAP (Simple Object Access Protocol)	Struttura più formale definita da WSDL (Web Services Description Language).	XML	Formale (WSDL).	WCF (Windows Communication Foundation).
GraphQL (Graph Query Language)	È il client a decidere quali dati leggere .	Richiesta: GraphQL + JSON; Risposta: JSON.	Formale definita da SDL (Schema Definition Language).	GraphQL.net, HotChocolate.

API Gateway

L'API Gateway è il controllore del traffico per le richieste ai microservizi. Le sue funzioni principali sono:

- **Mappatura degli endpoint:** mappa ciascun endpoint al microservizio specifico.
- **Catalogo:** funge da catalogo dei microservizi e delle loro API.
- **Versionamento:** permette il versionamento delle API anche con versioni diverse dei microservizi.
- **Deploy senza downtime:** facilita il deploy (rollout) di aggiornamenti in tre fasi (installazione nuova versione, aggiornamento del gateway, disinstallazione vecchia versione).
- **Discovery**
- **Load-Balancing**